Algoritmo Dígito por Dígito para Cálculo de Raiz Quadrada no ATmega328

Gabriel Luduvice Nunes Wolfovitch, João Pedro Nascimento Monteiro e
João Vítor de Araújo Costa de Oliveira
Departamento de Engenharia Elétrica e Computação (DEEC)
Universidade Federal da Bahia (UFBA)
Salvador, Brasil
{gabrielwolfovitch, joaovaco, monteirojoao}@ufba.br

***Resumo*—O problema proposto envolve o desenvolvimento do cálculo de uma raiz em um microcontrolador ATmega328. Como esse microcontrolador não possui uma unidade de ponto flutuante e há a impossibilidade de utilizar bibliotecas matemáticas vistas em C, este relatório apresenta um algoritmo que calcula a raiz quadrada binária pelo método dígito por dígito, feito inteiramente na linguagem Assembly. O método utiliza representação em ponto fixo, processando uma entrada de 16 bits (Q16.0) para gerar uma saída (Q8.6). A validação foi feita por meio de um teste prévio em C e testes de rastreamento, garantindo que o método dígito por dígito gera uma execução eficiente, ideal para o cálculo de variáveis.**

***Index Terms*—Raiz Quadrada, Assembly, Linguagem C, ATmega328.**

I. Introdução

O processamento em um sistema embarcado de instrumentação baseado em microcontroladores necessita realizar operações matemáticas básicas sobre as grandezas medidas, a exemplo da radiciação. Isto é, durante certas medições, o sistema recebe de sensores grandezas proporcionais ao quadrado de uma variável física. Para recuperar esta variável, é preciso calcular a raiz quadrada do valor medido.

No entanto, dispositivos como o ATmega328 não possuem funções nativas para o cálculo de raízes quadradas, necessitando implementações diretas. Frente às restrições relacionadas ao projeto, não é possível fazer uso de bibliotecas matemáticas, nem utilizar ponto flutuante.

Diante disso, a implementação via dígito por dígito faz uso de um algoritmo simples, baseado apenas em deslocamentos, somas e subtrações entre registradores, para realizar a operação desejada. Por se basear em agrupamentos e subdivisões, os erros associados são tão baixos quanto menor for a quantização em ponto fixo. O repositório com o código da implementação final pode ser encontrado em [1].

II. Técnica Utilizada e seu Funcionamento

A escolha da extração de raiz quadrada binária dígito a dígito foi baseada na sua simplicidade. Considerando operações em ponto fixo, este método é preferível por sua facilidade no acompanhamento e checagem, além de não necessitar de operações complexas.

Toda a lógica matemática é resolvida apenas através de:

- **Deslocamentos lógicos:** Operações já nativas das instruções do ATmega328, não necessitando grandes linhas de código para sua construção;
- **Subtrações:** Implementadas também nativamente entre os registradores com as instruções existentes;
- **Testes de Sinal:** Avaliação direta do bit de sinal do SREG para tomadas de decisão.

A. Fundamentação Matemática

Seja X o radicando inteiro de entrada e Q_k a aproximação parcial da raiz quadrada construída até a k-ésima iteração. A cada nova iteração, o algoritmo desloca a raiz parcial uma posição binária à esquerda e testa se o novo bit menos significativo $b_k \in \{0, 1\}$ deve ser igual a 1. Pela expansão binomial do quadrado de $(2Q_{k-1} + b_k)$, tem-se:

$$(2Q_{k-1} + b_k)^2 = 4Q_{k-1}^2 + (4Q_{k-1} + b_k)b_k \tag{1}$$

Se escolhermos $b_k = 1$, o incremento quadrático em relação ao quadrado parcial anterior ($4Q_{k-1}^2$) é dado pelo termo de teste T_k:

$$T_k = (2Q_{k-1} + 1)^2 - 4Q_{k-1}^2 = 4Q_{k-1} + 1 \tag{2}$$

Em representação binária, multiplicar Q_{k-1} por 4 e somar 1 equivale a deslocar Q_{k-1} dois bits para a esquerda e concatenar o par de bits 01 nas duas posições menos significativas:

$$T_k = (Q_{k-1} \ll 2) \mid 01_2 \tag{3}$$

Paralelamente, o radicando X é consumido da esquerda para a direita (do bit mais significativo para o menos significativo) em grupos de 2 bits por iteração, pois $(2^1)^2 = 2^2 = 4$. O resto parcial A_k é atualizado deslocando-se o resto anterior dois bits à esquerda e inserindo os 2 bits mais significativos atuais de X:

$$A_k = (A_{k-1} \ll 2) \mid X_{\text{MSB}[15:14]} \tag{4}$$

Em seguida, calcula-se a diferença de teste $S_k = A_k - T_k$:

- **Caso $S_k \geq 0$:** O resíduo A_k comporta a subtração. O novo bit da raiz é $b_k = 1$, o resto parcial é atualizado ($A_k \leftarrow S_k$) e a raiz parcial passa a ser $Q_k = (Q_{k-1} \ll 1) \mid 1$.
- **Caso $S_k < 0$:** O termo de teste excede o resíduo disponível. O novo bit da raiz é $b_k = 0$, o resto parcial

anterior A_k é preservado (caracterizando o método como *restaurador*) e a raiz parcial recebe apenas o deslocamento $Q_k = (Q_{k-1} \ll 1)$.

Mais detalhes acerca da demonstração matemática completa são dados em [2].

B. Extensão para Ponto Fixo (Q16.0 para Q8.6)

Para uma entrada X de 16 bits no formato inteiro Q16.0, as primeiras 8 iterações consomem integralmente os 8 pares de bits do radicando ($8 \times 2 = 16$ bits), produzindo os **8 bits da parte inteira** da raiz quadrada ($\lfloor \sqrt{X} \rfloor$).

Ao término da 8ª iteração, o registrador de entrada encontra-se completamente zerado. Contudo, caso ainda exista resto não nulo em A_8, a execução de iterações adicionais continua deslocando pares de bits `00` para dentro de A_k. Matematicamente, realizar F iterações extras após esgotar os bits de X equivale a multiplicar o radicando original por $4^F = 2^{2F}$ e extrair a raiz inteira:

$$Q_{8+F} = \left\lfloor \sqrt{X \cdot 2^{2F}} \right\rfloor = \left\lfloor 2^F \sqrt{X} \right\rfloor \tag{5}$$

Como a implementação executa um total de $N = 14$ iterações (8 inteiras +6 fracionárias, com $F = 6$), o valor binário final armazenado no registrador de saída está escalonado por $2^6 = 64$, correspondendo diretamente à representação em ponto fixo **Q8.6**.

III. Funcionamento do Algoritmo

Em linhas gerais, o algoritmo consiste em subdividir o número binário em subgrupos de 2 dígitos. A cada iteração, cada um desses grupos irá gerar um único dígito como resposta. Como a base binária permite apenas 0 ou 1, o dígito seguinte é decidido através da diferença entre o cálculo restante e o valor previsto. O algoritmo abaixo generaliza o exemplo mostrado em [3].

Algoritmo 1 Raiz quadrada inteira binária em ponto fixo

Entrada: Radicando X de $2n$ bits (sem sinal)
Saída: Raiz Q de n bits

1: $A \leftarrow 0$ ▷ acumulador
2: $Q \leftarrow 0$ ▷ raiz parcial
3: **para** $i \leftarrow 1$ **até** n **faça**
4: $A \leftarrow (A \ll 2) \mid X[2n-1:2n-2]$ ▷ 2 bits de X entram em A
5: $X \leftarrow X \ll 2$
6: $T \leftarrow A - \{Q, 01_2\}$ ▷ $\{Q, 01_2\} = (Q \ll 2) \mid 01_2$
7: $Q \leftarrow Q \ll 1$
8: **se** $T \geq 0$ **então**
9: $A \leftarrow T$
10: $Q[0] \leftarrow 1$
11: **end se**
12: **end para**
13: **retorne** Q

IV. Modelagem em Alto Nível: Linguagem C

A primeira etapa do projeto consistiu em validar o funcionamento do algoritmo de forma simplificada e rápida. Para isso, realizou-se uma implementação na linguagem C. O código abaixo demonstra a lógica principal.

```c
#include <stdint.h>
#include <stdio.h>

typedef uint8_t REG_t;

typedef struct ufix16 {
REG_t MSB;
REG_t LSB;
} ufix16_t;

float evaluate_ufix16_q88(const ufix16_t *num);
float evaluate_ufix16_q160(const ufix16_t *num);
ufix16_t take_sqrt(const ufix16_t *input);

int main(void) {
// assume input Q16.0 e output Q8.8
ufix16_t num = {0x0, 0xff};
printf("input is %f\n", evaluate_ufix16_q160(&num));
ufix16_t res = take_sqrt(&num);
printf("result is %f\n", evaluate_ufix16_q88(&res));

return 0;
}

float evaluate_ufix16_q88(const ufix16_t *num) {
uint16_t raw_value = ((uint16_t)num->MSB << 8) | num->LSB;
return (float)raw_value / 256.0f;
}

float evaluate_ufix16_q160(const ufix16_t *num) {
uint16_t raw_value = ((uint16_t)num->MSB << 8) | num->LSB;
return (float)raw_value;
}

ufix16_t take_sqrt(const ufix16_t *input) {
// Alinha a entrada de 16 bits (Q16.0) em um espaço de 32 bits.
// Isso efetivamente multiplica o número por 2^16, convertendo para Q16.16.
uint32_t val = ((uint32_t)input->MSB << 24) | ((uint32_t)input->LSB << 16);

uint32_t res = 0; // Acumulador da raiz quadrada (resultado)

// O 'bit' de teste começa na maior potência de 4 que cabe em 32 bits.
uint32_t bit = 1UL << 30;
// Alinha o 'bit' inicial ao tamanho real do valor de entrada
while (bit > val) {
    bit >>= 2;
}
// Laço de extração bit a bit
while (bit != 0) {
    if (val >= res + bit) {
    val = val - (res + bit);
    res = (res >> 1) + bit;
    } else {
    res >>= 1;
    }
    bit >>= 2;
}
// 'res' agora contém a raiz exata no formato Q8.8
ufix16_t result;
result.MSB = (res >> 8) & 0xFF;
result.LSB = res & 0xFF;
return result;
}
```

Implementação em C do algoritmo dígito por dígito.

A execução deste código em C auxiliou na compreensão mais abstrata do funcionamento geral do projeto. Assim, este foi adotado como base para a implementação final em Assembly.

V. Descrição do Sistema Final

A. Quantizações e Registradores

Para permitir uma maior amplitude de valores numéricos, utilizou-se 2 registradores para cada um dos termos descritos no algoritmo 1, além de 2 adicionais para a construção do valor $\{Q,\, 01_2\}$. Suas identificações e faixas de endereço são:

1) **Entrada X** (R19:18);
2) **Resposta Q** (R21:R20);
3) **Valor sendo trabalhado A** (R23:R22);
4) **Registrador do sinal T** (R25:R24);
5) **Resposta temporária** $\{Q,\, 01_2\}$ (R27:R26).

O sistema final pressupõe uma entrada já quantizada em Q16.0. A fim de cobrir uma ampla faixa dos números inteiros, decidimos interpretar a entrada como um número de 16 bits, dos quais todos são parte inteira. Assim, o código é capaz de calcular a raiz de um número n na forma: $\forall n \in \mathbb{Z}, n \in [0, 65535]$.

Além disso, a saída foi quantizada em Q8.6. O tamanho da parte inteira da saída, conforme mostrado no algoritmo 1, depende apenas do tamanho da entrada. A parte fracionária, para os casos de raízes não exatas, está relacionada com a quantidade de iterações realizadas após preenchimento da parte inteira. Um resumo dessas quantizações é dado abaixo na Tabela I

Tabela I
Especificações Numéricas de Entrada e Saída

Parâmetro Técnico	Entrada (`INPUT`)	Saída (`RESP`)
Notação Ponto Fixo	Q16.0 (sem sinal)	Q8.6 (sem sinal)
Largura Registrador	16 bits (`R19:R18`)	16 bits (`R21:R20`)
Bits Efetivos	16 bits ($[15:0]$)	14 bits ($[13:0]$)
Bits Int. / Frac.	16 int / 0 frac	8 int / 6 frac
Resolução (1 LSB)	$2^0 = 1$	$2^{-6} = 0{,}015625$
Valor Mínimo	0 (`0x0000`)	0,0 (`0x0000`)
Valor Máximo	65535 (`0xFFFF`)	255,984375 (`0x3FFF`)

Como cada iteração gera um dígito, o preenchimento da parte fracionária foi feito até o limite do *overflow* dos registradores intermediários do cálculo.

Para converter o valor binário bruto R_{raw} contido no registrador da resposta após a 14ª iteração para sua grandeza real estimada $\hat{y}$, divide-se o inteiro sem sinal por $2^6 = 64$:

$$\hat{y} = \frac{R_{\text{raw}}}{64} = (\texttt{RESP_HIGH} \ll 2) + \frac{\texttt{RESP_LOW}}{64} \tag{6}$$

B. Rotinas Principais

O código Assembly final tem um *loop* com duas rotinas principais, a saber:

1) SHIFT_DOIS;
2) CALCULA_SINAL.

Na primeira, 2 bits da entrada são carregado para os registradores que irão guardar o valor sendo atualmente trabalhado. Este processo é feito através de instruções com LSL e ROL, aproveitando o carry que possa ter surgido do primeiro deslocamento. Ambos são executados duas vezes em *loop*.

Na segunda etapa, ocorre a validação do sinal e a construção da resposta. O resultado atual é salvo em outro par de registradores através do MOVW, deslocado duas vezes e somado 1. Ou seja, este é o registrador $\{Q,\, 01_2\}$.

Em seguida, por meio de outro MOVW e subtrações sem e com carry (SUB e SBC), a resposta será comparada por meio dos registradores A e T. O controle dessa condicional é realizado pelo bit N do SREG, indicando se houve ou não um valor negativo durante a operação. Este processo é repetido até atingir uma saída na formatação escolhida.

O diagrama completo do sistema final pode ser encontrado em [1].

VI. Análise dos Resultados

Para validar o funcionamento do código em Assembly, realizou-se um teste com alguns casos particulares:

- Extremos: Valores no limite inferior e superior do intervalo possível de representação para números em ponto fixo Q16.0
- Raízes Exatas: Números que possuem raízes inteiras ("quadrados perfeitos")
- Raízes Inexatas: Números que não possuem raízes inteiras.

A Tabela II traz o resultado de alguns desses casos, mostrando qual foi a saída gerada a partir de cada entrada. A Tabela III traz mais exemplos, com os erros associados.

Tabela II
Casos de Teste da Raiz Quadrada (Entrada Q16, Saída Q8.6)

Entrada em Q16.0	Saída em Q8.6
0000000001111001 (121)	00001011.000000 (11,000000)
0000000000000000 (0)	00000000.000000 (0,000000)
1111111111111111 (65535)	11111111.111111 (255,984375)
0000000000000010 (2)	00000001.011010 (1,406250)

Para os casos em questão, podemos perceber que o algoritmo cumpriu com o esperado. Para os casos em que a entrada possuía raiz inteira, o cálculo foi exato, sem qualquer erro envolvido. Para os casos onde a raiz não era exata, pudemos validar que o erro esteve sempre limitado ao menor valor passível de representação ($2^{-6} \approx 0{,}015625$).

Isto pode ser mostrado analisando as propriedades das operações aplicadas pelo algoritmo. Como ele interrompe a extração na 6ª casa fracionária binária sem etapa adicional de arredondamento, a operação realizada é equivalente à função piso (*floor*) aplicada sobre a raiz escalonada:

$$\hat{y}(X) = \frac{\lfloor 2^6 \sqrt{X} \rfloor}{2^6} = \frac{\lfloor 64\sqrt{X} \rfloor}{64} \tag{7}$$

Pela desigualdade da função piso, sabe-se que $\lfloor 64\sqrt{X} \rfloor \leq 64\sqrt{X} < \lfloor 64\sqrt{X} \rfloor + 1$. Dividindo todos os termos por 64 e usando a Equação 7, obtém-se:

$$\hat{y}(X) \leq \sqrt{X} < \hat{y}(X) + 2^{-6} \tag{8}$$

Subtraindo $\sqrt{X}$ de todos os termos e multiplicando por -1, chega-se ao intervalo do erro $e(X) = \hat{y}(X) - \sqrt{X}$:

$$\begin{aligned} -2^{-6} &< \hat{y}(X) - \sqrt{X} \leq 0 \\ \Rightarrow \quad 0 &\leq \sqrt{X} - \hat{y}(X) < 0{,}015625 \end{aligned} \tag{9}$$

VII. Conclusão

Este trabalho demonstrou sucesso na implementação de um algoritmo de baixo nível para um microcontrolador. A escolha do método do dígito por dígito se mostrou uma boa abordagem para a arquitetura do ATmega328, extraindo o melhor possível do método de ponto fixo. A adoção do ponto fixo permitiu não apenas processar completamente os inteiros de 16 bits (Q16.0), mas também garantir precisão fracionária na resposta (Q8.6), chegando no limite de *overflow* dos registradores. Como mostrado nos testes de rastreamento, o controle de fluxo por meio da *flag* de sinal do registrador de status e o mapeamento direto das variáveis para os registradores (R18 a R27) garantiram um bom desempenho. Pode-se concluir que o código desenvolvido em Assembly atende aos requisitos solicitados, oferecendo um tempo de execução curto e um baixo uso de memória.

Apêndice A
Uso de Inteligência Artificial Generativa

Declaração

A equipe declara que a IAG foi utilizada nas etapas descritas abaixo. O detalhamento de cada uso, incluindo os *prompts* empregados, é apresentado a seguir.

Detalhamento

Uso 1
Etapa / local de uso: Demonstração Matemática do Algoritmo e Erros Associados
Modelo utilizado: *Claude Sonnet* 5.5 *(Anthropic)*
Prompt: *Demonstre matematicamente o funcionamento do algoritmo dígito por dígito em ponto fixo, considerando formatação Q16.0 na entrada e Q8.6 na saída. Prove também qual o limitante superior dos erros envolvidos*
Observações:

Uso 2
Etapa / local de uso: Formatação das referências bibliográficas
Modelo utilizado: *Claude Sonnet* 5.5 *(Anthropic)*
Prompt: *formate os links de referências abaixo em um ambiente thebibliography para latex em conformidade com o template IEEE conferences em português*
Observações: O *prompt* acompanhava a lista de referências mostrada ao final do relatório.

Uso 3
Etapa / local de uso: Generalização do código em Assembly
Modelo utilizado: *Claude Sonnet* 5.5 *(Anthropic)*
Prompt: *como esse código poderia ser escalado para lidar com números em uma faixa maior de representação? Por exemplo, em Q8.8?. Dê um direcionamento e ideias para uma implementação, sem enviar o código pronto.*
Observações: O *prompt* foi feito após já existir uma implementação funcional do algoritmo, porém para uma escala menor de representação, com apenas um registrador para a resposta. Acompanhava também o código existente até então.

Uso 4
Etapa / local de uso: Formatação e generalização do algoritmo para relatório
Modelo utilizado: *Claude Sonnet* 5.5 *(Anthropic)*
Prompt: *escreva isso genericamente em forma de um bloco latex de algoritmo para relatório*
Observações: Adaptado algumas vezes para adequar nomenclaturas. O *prompt* acompanhava a imagem de um exemplo presente em [3].

Uso 5
Etapa / local de uso: Formatação da seção anexa de IAG
Modelo utilizado: *Claude Sonnet* 5.5 *(Anthropic)*
Prompt: *faça um template para que eu possa colocar essa seção como um anexo do arquivo já existente no latex no template IEEE Conference. Alguns campos para o prompt, indicação de onde foi usado e o modelo usado*
Observações: O *prompt* incluía uma imagem com as orientações para a seção de IAG. Adaptado em relação à formatação.

Apêndice B
Tabela de Erros

Tabela III
Estimativa do Algoritmo (Q8.6) × Referência Real.

X **(Q16.0)**	**Hex (Q8.6)**	$\hat{y}$ **(Decimal)**	$\sqrt{X}$ **(Referência)**	$\lvert e_{\text{abs}} \rvert$	$\lvert e_{\text{rel}} \rvert$ **(%)**
0	0x0000	0,000000	0,000000	0,000000	0,000
2	0x005A	1,406250	1,414214	0,007964	0,563
10	0x00CA	3,156250	3,162278	0,006028	0,191
121	0x02C0	11,000000	11,000000	0,000000	0,000
255	0x03FD	15,953125	15,968719	0,015594	0,098
1000	0x07E7	31,609375	31,622777	0,013402	0,042
10000	0x1900	100,000000	100,000000	0,000000	0,000
16383	0x1FFF	127,984375	127,996094	0,011719	0,009

Referências

[1] "sismicro: Unidade 1," repositório GitHub. [Online]. Disponível em: https://github.com/JoaoNasMonteiro/sismicro/tree/main/u1. Acesso em: 18 out. 2026.

[2] "Square root algorithms: Digit By Digit Calculation," *Wikipedia, The Free Encyclopedia*. [Online]. Disponível em: https://en.wikipedia.org/wiki/Square_root_algorithms#Digit-by-digit_calculation. Acesso em: 18 out. 2026.

[3] W. Green, “Square root in Verilog,” *Project F*, 22 dez. 2020. [Online]. Disponível em: https://projectf.io/posts/square-root-in-verilog/. Acesso em: 18 out. 2026.
[4] Atmel Corporation, *ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash Datasheet*, Rev. 7810D-AVR-01/15, 2015.
[5] NCOT Technology, “Fixed point maths explained - Retro programming,” YouTube, 14 nov. 2021. [Vídeo]. Disponível em: https://www.youtube.com/watch?v=Is67DfCdvcE. Acesso em: 18 out. 2026.